

COUNCILia

Tareas manuales

Checklist de lo que debe hacer una persona fuera del código.

- Prioridades crítica a baja
- Anthropic y entorno local
- Supabase y migración SQL
- Vercel y DNS
- Correo, GitHub, Stripe
- Legal y usuarios pilotos
- Comandos útiles del repo

Tabla de contenidos

1 Resumen por prioridad

2 Crítica — desarrollo local y Claude

3 Alta — Supabase (persistencia, J1 / A6)

4 Alta — Vercel (deploy, A8)

5 Media — dominio y correo

6 Baja — pagos, legal, usuarios

7 Comandos útiles del repo

8 Contacto operativo (plantilla)

*Checklist consolidado de lo que **debe hacer una persona** fuera del código: cuentas, claves, DNS, legal y operaciones. El código del repo cubre el MVP técnico; estas acciones desbloquean **producción, persistencia y cumplimiento**.*

1. Resumen por prioridad

Prioridad	Bloquea	Acciones
Crítica	Desarrollo con LLM real, evaluación de prompts, producción con Claude	API key de Anthropic; variables en <code>.env.local</code>
Alta	Persistencia, auth, historial, deploy público	Supabase (proyecto + migración SQL); Vercel; variables en hosting
Media	Dominio público, correos, CI estricto	DNS; Resend (o similar); branch protection en GitHub
Baja	Pagos, legal formal, validación con usuarios	Stripe; revisión legal; reclutamiento y consentimientos

2. Crítica — desarrollo local y Claude

2.1 Clave de Anthropic (`ANTHROPIC_API_KEY`)

1. Entra en [Anthropic Console](#) con la cuenta del equipo.
2. **API Keys** → crear clave nueva (`sk-ant-api03-...`).
3. En tu máquina: copia `.env.example` a `.env.local` (no lo subas a git).
4. Añade: ```bash ANTHROPIC_API_KEY=sk-ant-api03-... ```
5. Opcional — modelo explícito: ```bash ANTHROPIC_MODEL=claude-3-5-sonnet-20241022 ```
6. Verificación: ```bash npm run test:anthropic ``` Debe imprimir una respuesta corta del modelo.

2.2 Variables mínimas para `npm run dev`

- `ANTHROPIC_API_KEY` — necesaria para `/api/sessions/*` y la pantalla **Sesión** con streaming real.
- `NEXT_PUBLIC_APP_URL` — opcional en local; recomendado `http://localhost:3000` para sitemap/robots coherentes.

2.3 CI local sin secretos

Para `npm run build` / CI sin claves, el proyecto admite:

```
SKIP_ENV_VALIDATION=1 npm run build
```

No uses `SKIP_ENV_VALIDATION=1` en producción real.

2.4 Cursor / TypeScript (errores fantasma)

Si el IDE muestra errores en rutas que ya no existen (p. ej. `councilia-scaffold/...`):

- `Cmd+Shift+P` → **TypeScript: Restart TS Server**

o **Developer: Reload Window**

3. Alta — Supabase (persistencia, J1 / A6)

3.1 Crear proyecto

1. [Supabase](#) → **New project** (región cercana a usuarios, p. ej. `us-east-1`).
2. Anota **Project URL** y **anon public key** (Settings → API).

3.2 Variables en `.env.local`

```
SUPABASE_URL=https://<ref>.supabase.co  
SUPABASE_ANON_KEY=<eyJ...>
```

3.3 Aplicar migración SQL

1. Dashboard → **SQL Editor** → nueva query.
2. Pega el contenido completo de `supabase/migrations/001_init.sql` del repo.
3. **Run**.
4. Comprueba **Authentication** → **Políticas**: tablas `users`, `councils`, `conversations`, `messages` con RLS activa.

Instrucciones ampliadas: [supabase/README.md](#) en el repo.

3.4 Siguiendo el paso técnico (cuando el proyecto exista)

El código incluye repos en memoria y un **factory** listo para enchufar el cliente real de Supabase ([@supabase/supabase-js](#)). Falta cablear `SupabaseRepos` y endpoints de guardado (I5) — eso lo hace el equipo en código una vez el proyecto y la migración estén vivos.

4. Alta — Vercel (deploy, A8)

1. [Vercel](#) → **Add New** → **Project** → conecta el repo [Councilia/Councilia](#).
 2. Framework: Next.js (auto).
 3. **Environment Variables** (Production, Preview y Development): - [ANTHROPIC_API_KEY](#) - [SUPABASE_URL](#), [SUPABASE_ANON_KEY](#) - [NEXT_PUBLIC_APP_URL](#) (p. ej. <https://dev.councilia.app> en preview/staging)
 4. Primer deploy; revisar logs si falla el build (en CI ya se usa [SKIP_ENV_VALIDATION](#) donde aplica).
-

5. Media — dominio y correo

5.1 Dominio ([councilia.app](#), A9)

1. Comprar dominio (Namecheap, Porkbun, Cloudflare Registrar, etc.).
2. En Vercel → proyecto → **Domains**: añadir [councilia.app](#) y [dev.councilia.app](#).
3. Aplicar los registros DNS que indique Vercel (apex [A/ALIAS](#), [CNAME](#) para subdominios).
4. HTTPS lo gestiona Vercel; comprobar redirección [www](#) → apex si aplica.

5.2 Correo transaccional (O4 — Resend u otro)

1. Cuenta en [Resend](#) (o proveedor equivalente).
2. Verificar dominio (SPF, DKIM) para enviar desde [@councilia.app](#).
3. API key en variables de entorno del backend (cuando exista el envío de mails).

5.3 GitHub — protección de rama (A7 estricto)

1. Repo → **Settings** → **Branches** → **Branch protection rules** para [main](#).
 2. Activar: *Require status checks*, *Require pull request*, bloquear force-push.
 3. Marcar el job de CI que ejecuta lint + typecheck + tests + build ([verify](#) o el nombre del workflow).
-

6. Baja — pagos, legal, usuarios

6.1 Stripe (N1–N5, H2)

1. [Stripe Dashboard](#) — empezar en **Test mode**.
2. Crear productos: Pro mensual y anual (USD y MXN según plan comercial).
3. Guardar Price IDs para Checkout y webhooks.

4. Webhook signing secret en `.env` cuando se implemente [I8](#).

6.2 Revisión legal (R1 — bloquea paywall serio, H2)

Contratar revisión por abogado/a de:

- `docs/10_terminos_y_condiciones.md`
- `docs/12_politica_de_privacidad.md`
- `docs/13_politica_de_cookies.md`

Sin versiones legales aprobadas, no conviene activar paywall agresivo en producción.

6.3 Validación con usuarios (S2, semanas 7–8)

- Reclutar 20–30 personas externas al equipo.
- Consentimiento informado por escrito (y grabación si aplica).
- Retención de datos de prueba acorde a política de privacidad.

7. Comandos útiles del repo

Comando	Uso
<code>npm install</code>	Dependencias
<code>npm run dev</code>	Servidor local http://localhost:3000
<code>npm run build</code>	Build de producción
<code>npm run lint / npm run typecheck / npm test</code>	Calidad
<code>npm run eval</code>	Evaluación de prompts contra Claude (requiere API key)
<code>python3 scripts/build_pdfs.py</code>	Regenera PDFs desde <code>docs/*.md</code> (requiere <code>pip3 install -r scripts/requirements-pdf.txt</code>)

8. Contacto operativo (plantilla)

- Email de producto (ejemplo en UI de soporte): `hola@councilia.app` — **crear buzón real** o sustituir por uno existente del equipo.

Este documento es la referencia humana para onboarding de operadores. La versión en PDF se genera con `python3 scripts/build_pdfs.py` una vez añadido el doc 14 al script de build.